

NPM.DiscoverComponentsAndInterfaces.ps1

SolarWinds SWIS PowerShell script — add multiple components (nodes) and discover their interfaces

```
# This script combines two building blocks from the SolarWinds SDK samples:
#
# o CRUD.AddNode.ps1
#   - adds a node (component) using CRUD operations (New-SwisObject) and
#     registers the standard set of pollers for it.
#
# o NPM.DiscoverAndAddInterfacesOnNode.ps1
#   - uses Orion.NPM.Interfaces.DiscoverInterfacesOnNode and
#     Orion.NPM.Interfaces.AddInterfacesOnNode (SWISv3 verbs, NPM only)
#     to discover and add the interfaces of a node.
#
# The result is a single script that walks a list of components (nodes),
# adds every one of them that does not already exist, and then discovers
# and adds their interfaces for monitoring.
#
# Please update the hostname/credential setup below to match your
# environment before running.
#
# The list of components (nodes) can be supplied via the -IPAddresses
# parameter, e.g.:
# .\NPM.DiscoverComponentsAndInterfaces.ps1 -IPAddresses "10.0.0.1","10.0.0.2","10.0.0.3"
# .\NPM.DiscoverComponentsAndInterfaces.ps1 -IPAddresses "10.0.0.1,10.0.0.2,10.0.0.3"
# If -IPAddresses is not supplied, the sample list below is used instead.

param(
    # One or more IP addresses to add/discover. Accepts either an array of
    # strings or a single comma-separated string.
    [string[]]$IPAddresses,

    # Shared settings applied to every IP address passed via -IPAddresses.
    [int]$EngineID = 1,
    [int]$SNMPVersion = 2
)

# Connect to SWIS
$hostname = "localhost"
$username = "admin"
$password = New-Object System.Security.SecureString
$cred = New-Object -typename System.Management.Automation.PSCredential -argumentlist $username, $password
$swis = Connect-Swis -host $hostname -cred $cred

# The list of components (nodes) to add and discover interfaces on.
if ($IPAddresses)
{
    # Build $components from the -IPAddresses parameter (supports both
    # "-IPAddresses ip1,ip2" and "-IPAddresses 'ip1,ip2'" invocation styles).
    $components = $IPAddresses `
        | ForEach-Object { $_.Split(',') } `
        | ForEach-Object { $_.Trim() } `
        | Where-Object { $_ } `
        | ForEach-Object { @{ IPAddress = $_; EngineID = $EngineID; SNMPVersion = $SNMPVersion; DNS = ""; SysName = "" } }
}
else
{
    # Fallback sample list. Add as many entries as needed.
    $components = @(
        @{ IPAddress = "10.0.0.1"; EngineID = 1; SNMPVersion = 2; DNS = ""; SysName = "" },
        @{ IPAddress = "10.0.0.2"; EngineID = 1; SNMPVersion = 2; DNS = ""; SysName = "" },
        @{ IPAddress = "10.0.0.3"; EngineID = 1; SNMPVersion = 2; DNS = ""; SysName = "" }
    )
}
```

```

}

function Add-Component
{
    param($component)

    $newNodeProps = @{
        IPAddress = $component.IPAddress;
        EngineID = $component.EngineID;

        # SNMP v2 specific
        ObjectSubType = "SNMP";
        SNMPVersion = $component.SNMPVersion;

        DNS = $component.DNS;
        SysName = $component.SysName;

        # === default values ===

        # EntityType = 'Orion.Nodes'
        # Caption = ''
        # DynamicIP = false
        # PollInterval = 120
        # RediscoveryInterval = 30
        # StatCollection = 10
    }

    $newNodeUri = New-SwisObject $swis -EntityType "Orion.Nodes" -Properties $newNodeProps
    $nodeProps = Get-SwisObject $swis -Uri $newNodeUri

    # register the standard set of pollers for the node
    $poller = @{
        NetObject = "N:" + $nodeProps["NodeID"];
        NetObjectType = "N";
        NetObjectID = $nodeProps["NodeID"];
    }

    foreach ($pollerType in @(
        "N.Status.ICMP.Native",
        "N.ResponseTime.ICMP.Native",
        "N.Details.SNMP.Generic",
        "N.Uptime.SNMP.Generic",
        "N.Cpu.SNMP.CiscoGen3",
        "N.Memory.SNMP.CiscoGen3"))
    {
        $poller["PollerType"] = $pollerType;
        New-SwisObject $swis -EntityType "Orion.Pollers" -Properties $poller | Out-Null
    }

    return $nodeProps["NodeID"]
}

function Add-DiscoveredInterfaces
{
    param($nodeId)

    # Discover interfaces on the node
    $discovered = Invoke-SwisVerb $swis Orion.NPM.Interfaces DiscoverInterfacesOnNode $nodeId

    if ($discovered.Result -ne "Succeed")
    {
        Write-Host " Interface discovery failed for node $nodeId."
        return
    }

    # Uncomment one of the following to limit the interfaces that get added:

```

```

# No. 1: Remove interfaces that are NOT ifType 6 (FastEthernet)
#$discovered.DiscoveredInterfaces.DiscoveredLiteInterface | ?{ $_.ifType -ne 6 } | %{ $discovered.DiscoveredInter

# No. 2: Remove interfaces that have a caption of 'lo' (Loopback)
#$discovered.DiscoveredInterfaces.DiscoveredLiteInterface | ?{ $_.Caption.InnerText -eq 'lo' } | %{ $discovered.D

$interfaceCount = $discovered.DiscoveredInterfaces.DiscoveredLiteInterface.Count

# Add the remaining interfaces
Invoke-SwisVerb $swis Orion.NPM.Interfaces AddInterfacesOnNode @($nodeId, $discovered.DiscoveredInterfaces, "AddD

Write-Host " Added $interfaceCount interface(s) for node $nodeId."
}

# Walk every component: add it (unless it already exists), then discover
# and add its interfaces.
foreach ($component in $components)
{
    Write-Host "Processing component $($component.IPAddress)..."

    try
    {
        $existing = Get-SwisData $swis "SELECT NodeID FROM Orion.Nodes WHERE IPAddress = @ip" @{ ip = $component.IPAD

        if ($existing)
        {
            $nodeId = $existing
            Write-Host " Node already exists (NodeID $nodeId), skipping add."
        }
        else
        {
            $nodeId = Add-Component $component
            Write-Host " Added node (NodeID $nodeId)."
        }
    }

    Add-DiscoveredInterfaces $nodeId
}
catch
{
    Write-Host " Failed to process $($component.IPAddress): $($_.Exception.Message)"
}
}

```